

מבחן במבוא למדעי המחשב – 67101, 67108

מועד ב' 21.4.16

מורי הקורס: אביב זהר ונעם ניסן

במבחן זה שני חלקים. יש לענות על כל 10 השאלות בחלק א', ולבחור 4 מתוך 6 השאלות בחלק ב' (אין לענות על יותר מ-4 שאלות בחלק ב').

יש לענות על כל השאלות בגוף השאלון – בתוך "קופסאות התשובה" בלבד.

חלק א' – שאלות סגורות (40% -- יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב-4 נק')

בחלק זה של המבחן יש לכתוב את התשובה הסופית בלבד בתוך המשבצת המתאימה. אין צורך לנמק, ונימוקים גם לא יקראו.

1. מה זמן הריצה של הפונקציה הבאה?

```
def funny(n):  
    if n <= 6:  
        return 7  
    return funny(n//8) + funny(4)
```

זמן הריצה של הפונקציה הוא

2. מה זמן הריצה של הפונקציה הבאה?

```
def loopy_loop(n):  
    for i in range(n):  
        for j in range(i):  
            if j*j > i:  
                break
```

זמן הריצה של הפונקציה הוא

3. מה מדפיס הקוד הבא:

```
def f(a):  
    a[0] = 7  
    b[0] = 8  
  
a = [2]  
b = [3]  
f(b)  
print("a=", a, "b=", b)
```

4. מה מדפיס הקוד הבא:

```
def f(x):  
    return 100  
  
f = lambda x: 1 if x < 2 else x + f(x-1)  
print(f(5))
```

5. מה ידפיס הקוד הבא:

```
class MyClass:

    y = 0

    def __init__(self,x):
        self.x = MyClass.y
        MyClass.y = x

    def __str__(self):
        return str(self.x)

one = MyClass(1)
two = MyClass(2)
three = MyClass(3)
print(MyClass.y,one,two,three)
```

6. מה יודפס ע"י הקוד הבא?

```
seq = [4,4,5,3,4,3,3,3,1,1,1]
u = []
x = [u.append(item) for item in seq if (not u) or u[-1] != item]
print(u)
```

7. נתונות 3 פונקציות:

```
def f1():
    a = []
    for j in range(100000): a.append(j*j)
    for j in range(100000): if 99999*j == a[j]: print("yes")

def f2():
    a = []
    for j in range(100000): a.append(j*j)
    for j in range(100000): if 99999*j in a: print("yes")

def f3():
    d = {}
    for j in range(100000): d[j] = j*j
    for j in range(100000): if 99999*j in d: print("yes")
```

סביר שהפונקציה שתרוץ הכי מהיר (בפועל) תהיה

וזאת שתרוץ הכי לאט תהיה

8. מה יודפס ע"י הקוד הבא?

```
def xyz():
    history = []
    def g(x):
        history.append(x)
        if x < len(history):
            return history[x]
    return g
```

```
f = xyz()
print(f(7), end=" ")
print(f(0), end=" ")
print(f(1), end=" ")
print(f(3), end=" ")
```

9. מה ידפיס הקוד הבא?

```
two_to_one = lambda g: (lambda y: g(y, y))
one_to_two = lambda f: (lambda x, y: f(x)+f(y))
h=one_to_two(two_to_one (lambda x, y: x*y))
print (h(3, 2))
```

10. מה ידפיס הקוד הבא?

```
def funky_fun(lst):
    tmp = list(map(lambda x,y: x+y, lst[1:],lst[:-1]))
    return [lst[0]] + tmp + [lst[-1]]

print(funky_fun([1]))
print(funky_fun(funky_fun([1])))
print(funky_fun(funky_fun(funky_fun([1]))))
```

חלק ב' – שאלות תכנות (60% – יש לענות רק על 4 מתוך 6 השאלות, כל שאלה 15 נק')

יש לענות על כל שאלה רק בתוך הקופסה שניתנה בדף.

אנו ממליצים להשתמש במחברת הטיוטא כדי לגבש את הפתרון ורק כשהוא מושלם להעתיקו לטופס המבחן. אין צורך בהערות או תיעוד במבחן זה.

יש לכתוב קוד בהיר ויעיל.

סמנו בטבלה מהן השאלות עליהן בחרתם לענות:

שאלה 1	שאלה 2	שאלה 3	שאלה 4	שאלה 5	שאלה 6

2. כתבו פונקציה `cartesian_product` שמקבלת רשימה של רשימות של אותיות ומחזירה רשימה שהינה המכפלה הקרטזית שלהן. כלומר את רשימת המילים השונות שאפשר ליצור על ידי לקיחת האות הראשונה מהרשימה הראשונה, את האות השניה מהרשימה השניה, וכו'.

```
cartesian_product(["a", "b"], ["c", "d", "e"]) =
    ["ac", "ad", "ae", "bc", "bd", "be"]
```

```
cartesian_product(["a"], ["b", "c"], ["d"]) = ["abd", "acd"]
```

```
cartesian_product(["a", "b"], [], ["c", "d"]) = []
```

```
cartesian_product([]) = [""]
```

הערות: אפשר להניח (בלי לבדוק) שהקלט תקין, כלומר מכיל רשימה של רשימות של אותיות. אפשר להניח גם שאף אות אינה מופיעה יותר מפעם אחת במחרוזת אחת. אין חשיבות לסדר המחרוזות ברשימת התוצאה (אך יש חשיבות לסדר האותיות בכל מחרוזת ברשימת התוצאה).

```
def cartesian_product(lst):
```


3. כתבו אובייקט שהינו מעין "רב-מילון": בו ניתן לשייך **ערכים רבים לכל מפתח**. הפעולות עליכן לתמוך בהן הן:

MultiDict()	יצירת "רב-מילון" חדש ללא מפתחות וללא ערכים:
add(key, val)	מוסיף את הערך החדש כמשויך למפתח זה:
has_val(key, val)	מחזיר ערך בוליאני האומר האם הערך הנתון משויך למפתח נתון:
	דוגמא:

```
m = MultiDict()
m.add("a", 1)
m.add("b", 2)
m.add("a", 3)

m.has_val("a", 1)      # returns True
m.has_val("a", 2)      # returns False
m.has_val("b", 1)      # returns False
m.has_val("a", 3)      # returns True
```

[illegible]

```
d = {0:0, 7:10, 20:30}
f = interpolate(d)
print(f(-1), f(5), f(7), f(18))
```

אפשר להניח שהמילון תמיד מכיל את המפתח 0. במקרה של X שהינו בדיוק באותו המרחק משתי נקודות במילון, מותר להחזיר כל אחד מהערכים של נקודות אלו.

```
def interpolate(d):
```

5. כתבו פונקציה `diff_iter` המקבלת איטרטור (שעובר על סדרת מספרים ארוכה) ומייצרת איטרטור חדש שעובר על סדרת ההפרשים שלה (כלומר על ההפרשים בין מספרים עוקבים בסדרה המקורית).

לדוגמא, עבור הסדרה

1, 1, 5, 7, 1, 3, 8, ...

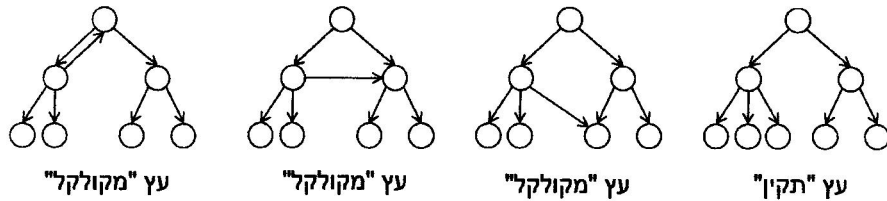
יוחזרו הערכים הבאים ע"י האיטרטור

$$0, 4, 2, -6, 2, 5, \dots$$

```
def diff_iter(some_iter):
```

6. נתונה המחלקה הבאה המייצגת צומת בעץ.

```
class TreeNode:
    def __init__(self, children, data):
        self.children = set(children)    #the set of child nodes
        self.data = data                #the data stored in the node
```



עץ הוא "תקין" אם לכל צומת יש מסלול יחיד המגיע אליה מהשורש. בעץ "מקולקל" (כלומר גרף שאינו עץ תקין) ישנן צמתים אליהן ניתן להגיע ביותר מדרך אחת. כתבו פונקציה `find_duplicate` שעבור עץ מקולקל מחזירה `TreeNode` כלשהו שניתן להגיע אליו מהשורש בשני מסלולים שונים, ועבור עץ תקין מחזירה `None`.

```
def find_duplicate(root):
```